Next Generation Databases

NoSQL, NewSQL, and Big Data

What every professional needs to know about the future of databases in a world of NoSQL and Big Data

Guy Harrison

Apress®

Next Generation Databases

NoSQL, NewSQL, and Big Data

Guy Harrison

Apress®

Next Generation Databases

ISBN-13 (pbk): 978-1-4842-1330-8

ISBN-13 (electronic): 978-1-4842-1329-2

Managing Director: Welmoed Spahr
Lead Editor: Jonathan Gennick
Development Editor: Douglas Pundick
Technical Reviewer: Stephane Faroult
Editorial Board: Steve Anglin, Pramila Balen, Louise Corrigan, Jim DeWolf, Jonathan Gennick, Robert Hutchinson, Celestin Suresh John, Michelle Lowman, James Markham, Susan McDermott, Matthew Moodie, Jeffrey Pepper, Douglas Pundick, Ben Renow-Clarke, Gwenan Spearing
Coordinating Editor: Jill Balzano
Copy Editor: Carole Berglie
Compositor: SPi Global
Indexer: SPi Global
Artist: SPi Global
Cover Designer: Anna Ishchenko

Distributed to the book trade worldwide by Springer Science+Business Media New York, 233 Spring Street, 6th Floor, New York, NY 10013. Phone 1-800-SPRINGER, fax (201) 348-4505, e-mail orders-ny@springer-sbm.com, or visit www.springer.com. Apress Media, LLC is a California LLC and the sole member (owner) is Springer Science + Business Media Finance Inc (SSBM Finance Inc). SSBM Finance Inc is a Delaware corporation.

For information on translations, please e-mail rights@apress.com, or visit www.apress.com.

Apress and friends of ED books may be purchased in bulk for academic, corporate, or promotional use. eBook versions and licenses are also available for most titles. For more information, reference our Special Bulk Sales–eBook Licensing web page at www.apress.com/bulk-sales.

Any source code or other supplementary material referenced by the author in this text is available to readers at www.apress.com. For detailed information about how to locate your book's source code, go to www.apress.com/source-code/.

To Catherine Maree Arnold (1981-2010)

Contents at a Glance

Contents

About the Author

Guy Harrison started working as a database developer in the mid-1980s. He has written numerous books on database development and performance optimization. He joined Quest Software (now part of Dell) in 2000, and currently leads the team that develops the Toad, Spotlight, and Shareplex product families.

Guy lives in Melbourne Australia, with his wife, a variable number of adult children, a cat, three dogs, and a giant killer rabbit.

Guy can be reached at guy.a.harrison@gmail.com at http://guyharrison.net and is @guyharrison on Twitter.

About the Technical Reviewer

Stéphane Faroult is a French consultant who first discovered relational databases and the SQL language 30 years ago. Stéphane joined Oracle France in its early days (after a brief spell with IBM and a period of time teaching at the University of Ottawa), and developed an interest in performance and tuning topics, on which he soon started writing training courses. After leaving Oracle in 1988, Stéphane briefly tried going straight and did a bit of operational research, but after only a year he succumbed again to the allure of relational databases. He is currently visiting faculty in the Computing and Information Science Department at Kansas State University. For his sins, Stéphane has been performing database consultancy continuously ever since; he founded RoughSea, Ltd. in 1998. In recent years, Stéphane has had a growing interest in education, which has taken various forms, including books (*The Art of SQL*, soon followed *by Refactoring SQL Applications*, both published by O'Reilly) and more recently a textbook (*SQL Success*, published by RoughSea), a series of seminars in Asia, and video tutorials (`www.youtube.com/user/roughsealtd`).

Acknowledgements

I'd like to thank everyone at Apress who helped in the production of this book, in particular lead editor Jonathan Gennick, coordinating editor Jill Balzano, and development editor Douglas Pundick. I'd like to especially thank Stéphane Faroult, who provided outstanding technical review feedback. Rarely have I worked with a reviewer of the caliber of Stéphane, and his comments were invaluable.

As always, thanks to my family—Jenni, Chris, Kate, Mike, and Willie—for providing the emotional support and understanding necessary to take on this project.

This book is dedicated to the memory of Catherine Maree Arnold, our beloved niece who passed away in 2010. I met Catherine when she was five years old; the first time I met her, she introduced me to the magic of Roald Dahl's *The Twits*. The last time I saw her, she explained modern DNA sequencing techniques and told me about her ambition to save species from extinction. She was one of the smartest, funniest, and kindest human beings you could hope to meet. When my first book was published in 1987, she jokingly insisted that I should dedicate it to her, so it's fitting that I dedicate this book to her memory.

—Guy Harrison
Melbourne, Australia
December 2015

PART I

Next Generation Databases

CHAPTER 1

Three Database Revolutions

Fantasy. Lunacy.

All revolutions are, until they happen, then they are historical inevitabilities.

— David Mitchell, Cloud Atlas

We're still in the first minutes of the first day of the Internet revolution.

—Scott Cook

This book is about a third revolution in database technology. The first revolution was driven by the emergence of the electronic computer, and the second revolution by the emergence of the relational database. The third revolution has resulted in an explosion of nonrelational database alternatives driven by the demands of modern applications that require global scope and continuous availability. In this chapter we'll provide an overview of these three waves of database technologies and discuss the market and technology forces leading to today's next generation databases.

Figure 1-1 shows a simple timeline of major database releases.

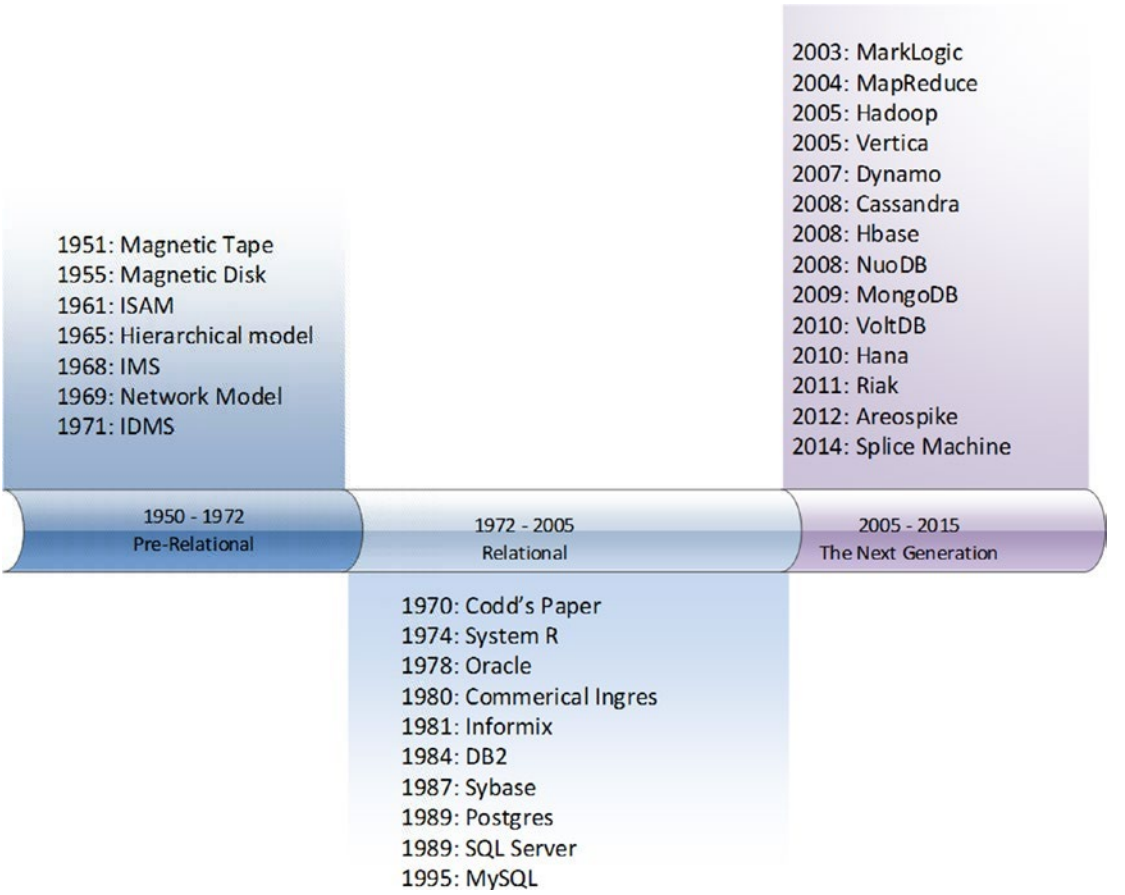

Figure 1-1. *Timeline of major database releases and innovations*

Figure 1-1 illustrates three major eras in database technology. In the 20 years following the widespread adoption of electronic computers, a range of increasingly sophisticated database systems emerged. Shortly after the definition of the relational model in 1970, almost every significant database system shared a common architecture. The three pillars of this architecture were the relational model, ACID transactions, and the SQL language.

However, starting around 2008, an explosion of new database systems occurred, and none of these adhered to the traditional relational implementations. These new database systems are the subject of this book, and this chapter will show how the preceding waves of database technologies led to this next generation of database systems.

Early Database Systems

Wikipedia defines a *database* as an "organized collection of data." Although the term *database* entered our vocabulary only in the late 1960s, collecting and organizing data has been an integral factor in the development of human civilization and technology. Books—especially those with a strongly enforced structure such as dictionaries and encyclopedias—represent datasets in physical form. Libraries and other indexed archives of information represent preindustrial equivalents of modern database systems.

We can also see the genesis of the digital database in the adoption of punched cards and other physical media that could store information in a form that could be processed mechanically. In the 19th century, loom cards were used to "program" fabric looms to generate complex fabric patterns, while tabulating machines used punched cards to produce census statistics, and player pianos used perforated paper strips that represented melodies. Figure 1-2 shows a Hollerith tabulating machine being used to process the U.S. census in 1890.

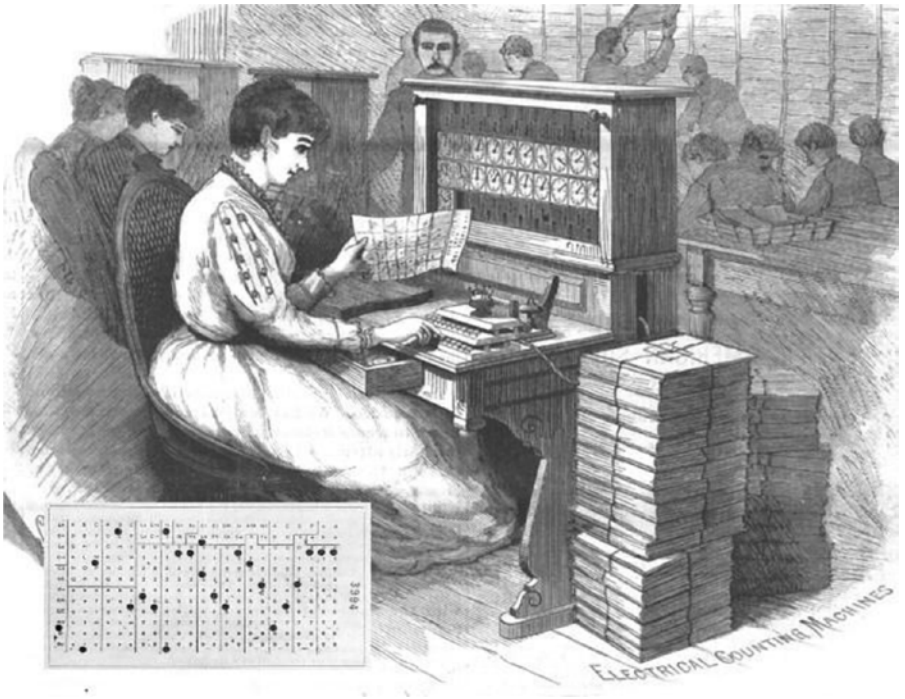

Figure 1-2. *Tabulating machines and punched cards used to process 1890 U.S. census*

The emergence of electronic computers following the Second World War represented the first revolution in databases. Some early digital computers were created to perform purely mathematical functions—calculating ballistic tables, for instance. But equally as often they were intended to operate on and manipulate data, such as processing encrypted Axis military communications.

Early "databases" used paper tape initially and eventually magnetic tape to store data sequentially. While it was possible to "fast forward" and "rewind" through these datasets, it was not until the emergence of the spinning magnetic disk in the mid-1950s that direct high-speed access to individual records became possible. Direct access allowed fast access to any item within a file of any size. The development of indexing methods such as ISAM (Index Sequential Access Method) made fast record-oriented access feasible and consequently allowed for the birth of the first OLTP (On-line Transaction Processing) computer systems.

ISAM and similar indexing structures powered the first electronic databases. However, these were completely under the control of the application—there were databases but no *Database Management Systems* (*DBMS*).

The First Database Revolution

Requiring every application to write its own data handling code was clearly a productivity issue: every application had to reinvent the database wheel. Furthermore, errors in application data handling code led inevitably to corrupted data. Allowing multiple users to concurrently access or change data without logically or physically corrupting the data requires sophisticated coding. Finally, optimization of data access through caching, pre-fetch, and other techniques required complicated and specialized algorithms that could not easily be duplicated in each application.

Therefore, it became desirable to externalize database handling logic from the application in a separate code base. This layer—the Database Management System, or DBMS—would minimize programmer overhead and ensure the performance and integrity of data access routines.

Early database systems enforced both a schema (a definition of the structure of the data within the database) and an access path (a fixed means of navigating from one record to another). For instance, the DBMS might have a definition of a CUSTOMER and an ORDER together with a defined access path that allowed you to retrieve the orders associated with a particular customer or the customer associated with a specific order.

These first-generation databases ran exclusively on the mainframe computer systems of the day — largely IBM mainframes. By the early 1970s, two major models of DBMS were competing for dominance. The *network model* was formalized by the CODASYL standard and implemented in databases such as IDMS, while the *hierarchical model* provided a somewhat simpler approach as was most notably found in IBM's IMS (Information Management System). Figure 1-3 provides a comparison of these databases' structural representation of data.

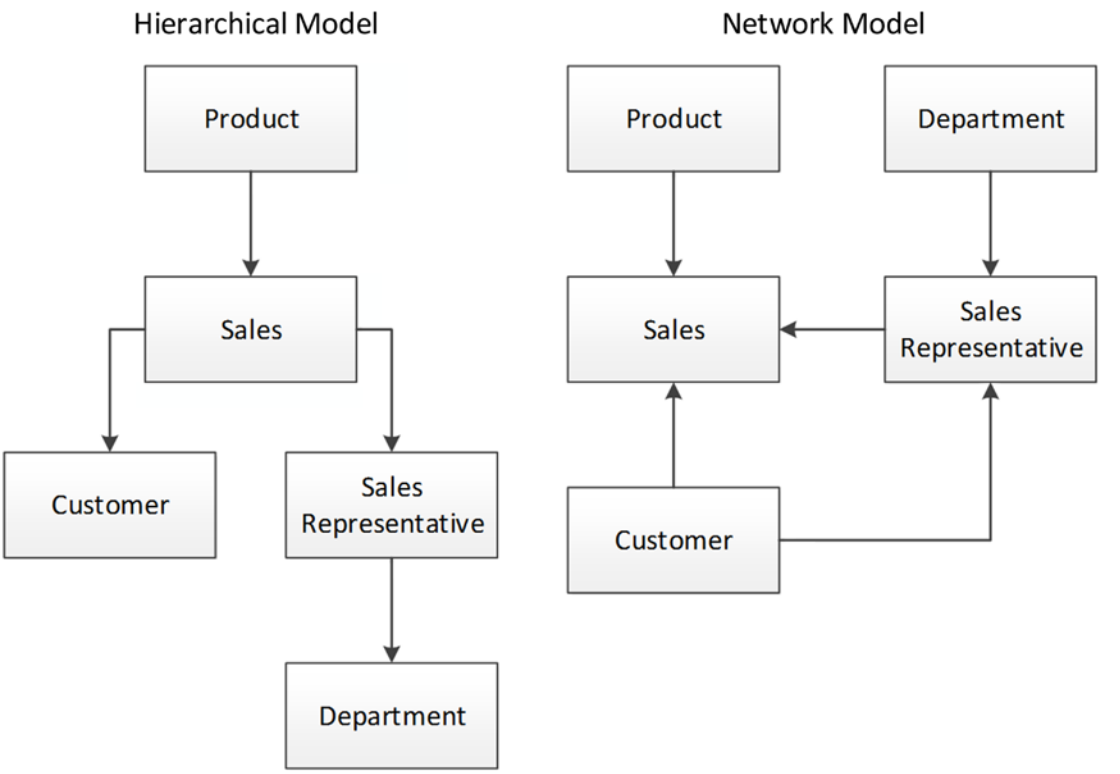

Figure 1-3. *Hierarchical and network database models*

■ **Note** These early systems are often described as "navigational" in nature because you must navigate from one object to another using pointers or links. For instance, to find an order in a hierarchical database, it may be necessary to first locate the customer, then follow the link to the customer's orders.

Hierarchical and network database systems dominated during the era of mainframe computing and powered the vast majority of computer applications up until the late 1970s. However, these systems had several notable drawbacks.

First, the navigational databases were extremely inflexible in terms of data structure and query capabilities. Generally only queries that could be anticipated during the initial design phase were possible, and it was extremely difficult to add new data elements to an existing system.

Second, the database systems were centered on record at a time transaction processing—what we today refer to as CRUD (Create, Read, Update, Delete). Query operations, especially the sort of complex analytic queries that we today associate with business intelligence, required complex coding. The business demands for analytic-style reports grew rapidly as computer systems became increasingly integrated with business processes. As a result, most IT departments found themselves with huge backlogs of report requests and a whole generation of computer programmers writing repetitive COBOL report code.

The Second Database Revolution

Arguably, no single person has had more influence over database technology than Edgar Codd. Codd received a mathematics degree from Oxford shortly after the Second World War and subsequently immigrated to the United States, where he worked for IBM on and off from 1949 onwards. Codd worked as a "programming mathematician" (ah, those were the days) and worked on some of IBM's very first commercial electronic computers.

In the late 1960s, Codd was working at an IBM laboratory in San Jose, California. Codd was very familiar with the databases of the day, and he harbored significant reservations about their design. In particular, he felt that:

- **Existing databases were too hard to use**. Databases of the day could only be accessed by people with specialized programming skills.
- **Existing databases lacked a theoretical foundation**. Codd's mathematical background encouraged him to think about data in terms of formal structures and logical operations; he regarded existing databases as using arbitrary representations that did not ensure logical consistency or provide the ability to deal with missing information.
- **Existing databases mixed logical and physical implementations**. The representation of data in existing databases matched the format of the physical storage in the database, rather than a logical representation of the data that could be comprehended by a nontechnical user.

Codd published an internal IBM paper outlining his ideas for a more formalized model for database systems, which then led to his 1970 paper "A Relational Model of Data for Large Shared Data Banks."[1] This classic paper contained the core ideas that defined the *relational database model* that became the most significant—almost universal—model for database systems for a generation.

Relational theory

The intricacies of relational database theory can be complex and are beyond the scope of this introduction. However, at its essence, the relational model describes how a given set of data should be presented to the user, rather than how it should be stored on disk or in memory. Key concepts of the relational model include:

- **Tuples**, an unordered set of **attribute** values. In an actual database system, a tuple corresponds to a row, and an attribute to a column value.
- A **relation**, which is a collection of distinct tuples and corresponds to a table in relational database implementations.
- **Constraints**, which enforce consistency of the database. Key constraints are used to identify tuples and relationships between tuples.
- **Operations** on relations such as joins, projections, unions, and so on. These operations always return relations. In practice, this means that a query on a table returns data in a tabular format.

A row in a table should be identifiable and efficiently accessed by a unique key value, and every column in that row must be dependent on that key value and no other identifier. Arrays and other structures that contain nested information are, therefore, not directly supported.

Levels of conformance to the relational model are described in the various "*normal forms*." *Third normal form* is the most common level. Database practitioners typically remember the definition of third normal form by remembering that all non-key attributes must be dependent on "the key, the whole key, and nothing but the key—So Help Me Codd"![2]

Figure 1-4 provides an example of normalization: the data on the left represents a fairly simple collection of data. However, it contains redundancy in student and test names, and the use of a repeating set of attributes for the test answers is dubious (while possibly within relational form, it implies that each test has the same number of questions and makes certain operations difficult). The five tables on the right represent a normalized representation of this data.

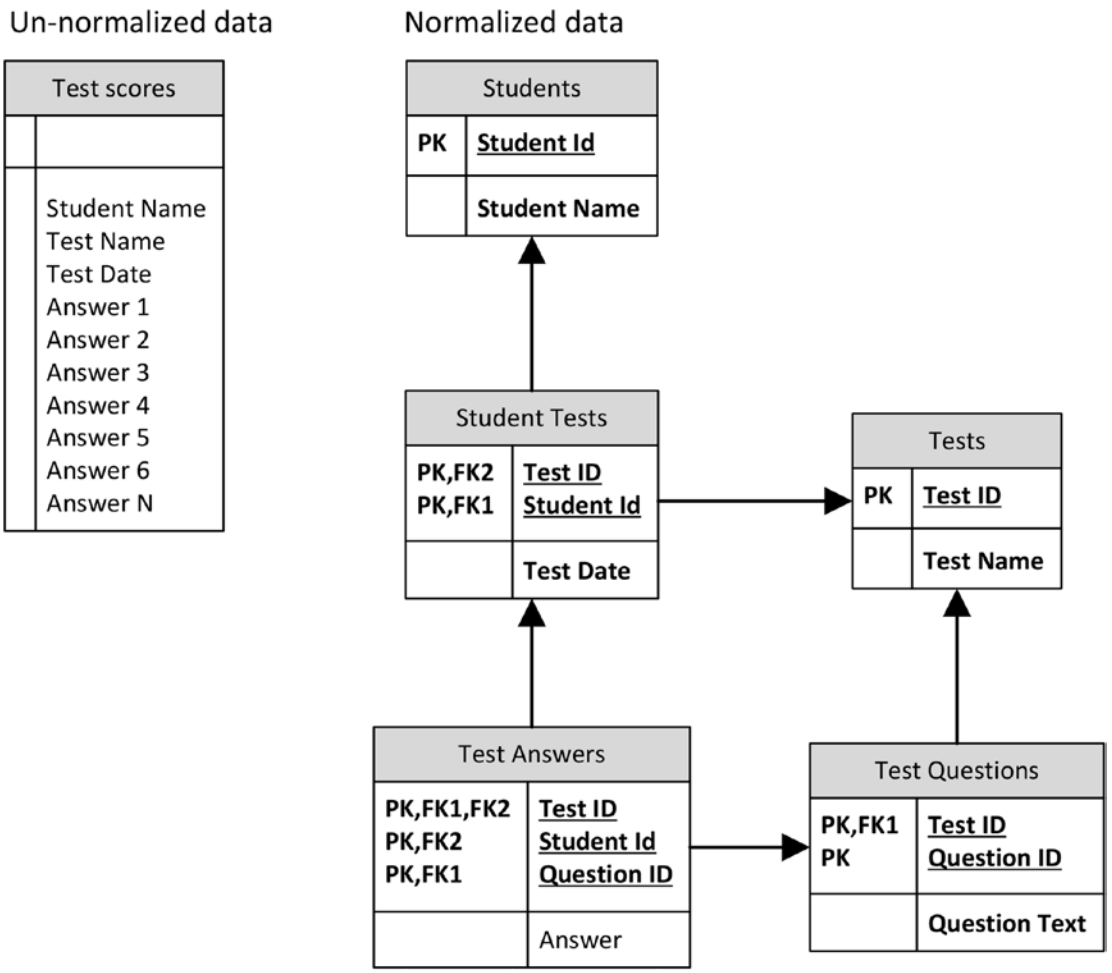

Figure 1-4. *Normalized and un-normalized data*

Transaction Models

The relational model does not itself define the way in which the database handles concurrent data change requests. These changes—generally referred to as database *transactions*—raise issues for all database systems because of the need to ensure consistency and integrity of data.

Jim Gray defined the most widely accepted transaction model in the late 1970s. As he put it, "A transaction is a transformation of state which has the properties of atomicity (all or nothing), durability (effects survive failures) and consistency (a correct transformation)."[3] This soon became popularized as *ACID transactions*: Atomic, Consistent, Independent, and Durable. An ACID transaction should be:

- **Atomic**: The transaction is indivisible—either all the statements in the transaction are applied to the database or none are.
- **Consistent**: The database remains in a consistent state before and after transaction execution.